Name : Vidhi Patel
Roll Number : 2401MC02
Date : 03/11/2026

OS Assignment 10 Report: Memory Management, Virtual Memory, Paging, and Page Replacement

1. System Modifications Overview

All kernel-level changes were implemented in xv6-riscv (64-bit architecture using uint64 types). The primary files modified in the kernel and user space include:

- **kernel/kalloc.c**: Added tracking variables (`total_allocated`, `total_freed`) to `struct kmem` guarded by `kmem.lock`. Updated `kalloc()` and `kfree()` to increment these counters. Added `get_memstats()` to calculate total, freed, currently allocated, and available physical memory.
- **kernel/vm.c**: Implemented `vtop_translate()` using `walk()` to convert virtual addresses to physical frame addresses; added `get_pageinfo()` to inspect page table entry flags (`PTE_V`, `PTE_W`, `PTE_U`); implemented `set_page_readonly()` to clear the `PTE_W` bit and flush the TLB using `sfence_vma()`.
- **kernel/defs.h & kernel/sysproc.c**: Added prototypes and kernel wrapper handlers (`sys_memstats`, `sys_vtop`, `sys_pageinfo`, `sys_setreadonly`).
- **kernel/syscall.h & kernel/syscall.c**: Registered unique system call numbers (22 to 25) and added mapping entries in the `syscalls[]` dispatch table.
- **user/user.h & user/usys.pl**: Added user-space stubs and prototypes to expose system calls to user programs.

2. Technical Implementation Details & Observations

Question 1: Memory Tracking (memtest.c)

- **Implementation**: Maintained counters for physical memory allocation. Evaluated active physical memory usage using: `Currently Allocated Pages = Total System Physical Pages - Available Free Pages`.
- **Observation**: Allocating 10 pages (40,960 bytes) via `sbrk(40960)` increased Total Allocated Pages and Currently Alloc Pages by 10, while decreasing Available Free Pages by 10. Calling `sbrk(-40960)` incremented Total Freed Pages by 10 and restored Currently Alloc Pages and Available Free Pages back to their baseline state.

Question 2: Address Translation (vttest.c)

- **Implementation**: Evaluated virtual-to-physical address mapping by walking the three-level page table using `walk()`. Extracted the Physical Frame Address via `PTE2PA(*pte)` and combined it with the lower 12-bit offset (`va & 0xFFF`).
- **Observation**: Valid virtual addresses produced exact physical address matches preserving offset values. Queries to addresses beyond `MAXVA` or unmapped user memory returned -1 and logged warning diagnostics.

Question 3: Page Protection & Fault Handling (pagetest.c)

- **Implementation**: `get_pageinfo()` inspected PTE flags. `set_page_readonly()` modified the page table entry in place by clearing `PTE_W` and calling `sfence_vma()` to flush TLB entries.
- **Observation**: Inspected flags initially showed writable status (W: 1). After executing `setreadonly()`, flags updated to W: 0. A subsequent write operation to the protected page immediately triggered a hardware Store Page Fault (`scause 0xf` / trap 15) at `stval`, causing the kernel to safely terminate the process (`usertrap`).

3. Page Replacement Algorithm Performance Analysis (Question 4)

Performance Comparison Table

Algorithm	Frame Count	Total Page Faults	Total Page Hits	Hit Ratio (%)	Page Fault Ratio (%)
FIFO	3	15	5	25%	75%
FIFO	4	10	10	50%	50%
FIFO	5	7	13	65%	35%
LRU	3	12	8	40%	60%
LRU	4	8	12	60%	40%
LRU	5	7	13	65%	35%
Optimal	3	9	11	55%	45%
Optimal	4	8	12	60%	40%
Optimal	5	7	13	65%	35%

Key Observations & Insights

- **Optimal Algorithm Superiority**: Optimal achieves the lowest number of page faults (9 faults with 3 frames) because it utilizes future knowledge of the reference string to evict the page that will not be needed for the longest time in the future.
- **LRU vs. FIFO**: LRU outperforms FIFO on constrained frame allocations (12 vs. 15 faults for 3 frames) because it leverages temporal locality (pages referenced recently are likely to be referenced again soon), whereas FIFO evicts pages strictly by arrival order regardless of access frequency.
- **Impact of Memory Frame Capacity**: Increasing frame capacity from 3 to 5 reduces page fault rates across all algorithms. At 5 frames, all three algorithms converge to 7 page faults (the compulsory cold-start faults required to load distinct pages into memory).